

1 How to target specific hyperlinks inside a list?

👉 Answer:

“In CSS, I can target anchor tags inside list items using descendant selectors like `li a`, but for production-level code I prefer using semantic classes for better scalability and maintainability. If positional targeting is required, I can use structural pseudo-classes like `:nth-child()`. However, relying purely on position is not recommended for dynamic content.”

Why?

- Improves maintainability
 - Avoids breaking styles when DOM structure changes
-

2 How to give fixed 500×500 dimensions using media query?

👉 Answer:

“I use media queries to apply fixed dimensions conditionally based on viewport size. For example, using `@media (min-width:768px)` ensures the fixed size applies only on larger screens. However, in responsive design, I generally prefer flexible units like `rem`, `%`, or `vw/vh` instead of hard-coded pixel values.”

Important:

- Fixed sizes can break responsiveness
 - Use only when layout demands strict dimensions
-

3 How many ways to apply CSS?

👉 Answer:

“There are three ways: Inline, Internal, and External CSS. In scalable applications, we primarily use External CSS for separation of concerns. Inline CSS has highest specificity but should be avoided because it reduces reusability and maintainability.”

Best Practice:

- Follow Separation of Concerns principle
 - Keep styling separate from structure
-

4 What is CSS Priority / Specificity?

👉 Answer:

“CSS specificity determines which rule gets applied when multiple rules target the same element. The hierarchy is Inline > ID > Class/Attribute/Pseudo-class > Element. Additionally, **!important** overrides normal specificity but should be avoided unless absolutely necessary.”

Formula:

Inline (1000)

ID (100)

Class (10)

Element (1)

5 What is Visibility?

👉 Answer:

“The **visibility** property controls whether an element is visible while preserving layout space. Unlike **display: none**, which removes the element from document flow, **visibility: hidden** keeps the layout intact. It is useful in UI transitions and toggling states without layout shifts.”

6 What are Formatting Elements?

👉 Answer:

“Formatting elements define text presentation and semantic meaning. For example, `<strong>` and `<em>` add semantic importance, which improves accessibility and SEO, unlike `<b>` and `<i>` which are purely visual.”

Experienced Insight:

Use semantic tags for accessibility and screen readers.

7 What is CSS Grid?

👉 Answer:

“CSS Grid is a two-dimensional layout system designed for complex layouts involving rows and columns. It allows explicit and implicit grid placement, supports fractional units (`fr`), and provides powerful alignment control. Compared to Flexbox, Grid is better suited for overall page layouts.”

When to use Grid:

- Dashboard layouts
 - Complex UI structure
 - Multi-column designs
-

8 How to target specific list element?

👉 Answer:

“In production-level applications, I prefer targeting elements using classes instead of structural selectors. Structural selectors like `:nth-child()` are acceptable for static content but may cause issues when DOM changes dynamically.”

9 How to target class name in JavaScript?

👉 Answer:

“I use `document.querySelector()` or `querySelectorAll()` because they support full CSS selector syntax and provide better flexibility compared to `getElementsByClassName()`. In modern applications, DOM manipulation is often handled via frameworks like React instead of direct DOM access.”

10 Background Properties?

👉 Answer:

“Background properties allow control over layering, positioning, and rendering of background visuals. In modern UI development, `background-size: cover` is widely used for hero sections, while `background-position` ensures proper alignment. Multiple backgrounds can also be layered using comma-separated values.”

11 Position Properties?

👉 Answer:

“The `position` property defines how elements are positioned in the document flow. `relative` creates a positioning context, `absolute` positions relative to nearest positioned ancestor, `fixed` is viewport-based, and `sticky` behaves as hybrid relative/fixed during scroll.”

Important Concept:

Absolute depends on nearest positioned ancestor.

12 How to give background color?

👉 Answer:

“Background color can be applied using `background-color` property with values like HEX, RGB, RGBA, or HSL. In professional UI development, RGBA and HSL are preferred when transparency or dynamic theming is required.”

1 Display Properties

◆ What is Display?

The `display` property controls **how an element behaves in layout**.

◆ Important Values

1. `block`

- Takes full width
- Starts on new line
- Can set width & height

Examples: `div`, `p`, `h1`

`display: block;`

2. `inline`

- Takes only required width
- Does NOT start new line

- Cannot set width & height

Examples: `span`, `a`

`display: inline;`

3. `inline-block`

- Inline behavior
- But width & height can be set

`display: inline-block;`

4. `none`

- Completely removes element from layout

`display: none;`

5. `flex`

- Enables Flexbox layout

`display: flex;`

6. `grid`

- Enables Grid layout

`display: grid;`

2 Relative Units (rem, em, px, %)

◆ px (Pixel)

- Fixed unit
- Not responsive

font-size: 16px;

◆ em

- Relative to **parent font-size**

font-size: 2em;

If parent is 16px → 2em = 32px

⚠ Problem: Nested elements compound.

◆ rem (Root em)

- Relative to root `<html>` font-size
- More predictable than em

font-size: 2rem;

If html = 16px → 2rem = 32px

✅ Best practice for scalable UI.

◆ Percentage (%)

- Relative to parent element size

width: 50%;

3 Min-Width & Max-Width

Used to control responsive limits.

```
min-width: 300px;  
max-width: 1200px;
```

Why Important?

- Prevent layout breaking
- Maintain responsiveness
- Used in containers

Example:

```
width: 100%;  
max-width: 1200px;  
margin: auto;
```

4 Position and Its Properties

Controls element placement.

◆ static (default)

- Normal flow
-

◆ relative

- Moves relative to original position

- Keeps space reserved

position: relative;
top: 10px;

◆ absolute

- Positioned relative to nearest positioned ancestor
- Removed from normal flow

position: absolute;
top: 0;
left: 0;

◆ fixed

- Fixed relative to viewport
- Doesn't scroll

position: fixed;

◆ sticky

- Acts relative until threshold
- Then becomes fixed

position: sticky;
top: 0;

5 Box Model

Every element is a box:

Margin
Border
Padding
Content

Formula:

Total Width = Content + Padding + Border + Margin

◆ box-sizing

content-box (default)

Width = content only

border-box (recommended)

Width includes padding + border

box-sizing: border-box;

6 Flexbox (1D Layout)

Used for row OR column layouts.

display: flex;

◆ Main Axis vs Cross Axis

- `flex-direction: row` → main axis horizontal
 - `flex-direction: column` → main axis vertical
-

◆ Important Properties

Parent Properties:

```
justify-content: center; /* main axis */  
align-items: center; /* cross axis */  
flex-wrap: wrap;  
gap: 10px;
```

Child Properties:

```
flex-grow: 1;  
flex-shrink: 1;  
flex-basis: 200px;
```

7 Grid Basics (2D Layout)

Used for rows AND columns.

```
display: grid;
```

◆ Define Columns

```
grid-template-columns: repeat(3, 1fr);
```

◆ Define Rows

```
grid-template-rows: 100px 200px;
```

◆ Gap

```
gap: 20px;
```

◆ Position Item

```
grid-column: 1 / 3;  
grid-row: 1 / 2;
```

8 Animation Syntax and Properties

◆ Basic Structure

```
@keyframes slide {  
  from { transform: translateX(0); }  
  to { transform: translateX(100px); }  
}  
  
.box {  
  animation: slide 2s ease-in-out 1s infinite alternate;  
}
```

◆ Animation Properties

Property	Meaning
animation-name	Keyframe name
animation-duration	Time
animation-timing-function	ease, linear
animation-delay	Start delay
animation-iteration-count	infinite or number
animation-direction	normal, alternate

9 Media Queries

Used for responsive design.

```
@media (max-width: 768px) {  
  body {  
    background: red;  
  }  
}
```

◆ Common Breakpoints

- 320px → Mobile
 - 768px → Tablet
 - 1024px → Laptop
 - 1200px+ → Desktop
-

◆ Types

@media (min-width: 768px)

@media (max-width: 768px)

@media (orientation: landscape)

Flexbox vs Grid (Easy & Clear)

Feature	Flexbox	Grid
Layout Type	1D (One Direction)	2D (Rows + Columns)
Controls	Row OR Column	Row AND Column
Best For	Aligning items in a line	Full page layouts
Complexity	Simple	More powerful
Use Case	Navbar, buttons, cards	Dashboard, gallery, layout structure

◆ 1. JavaScript Basics

JavaScript is a **scripting language** used to make websites interactive. It runs in the browser and also on servers (Node.js).

◆ JavaScript Data Types

✓ 1. Primitive Data Types

Type	Example	Common Methods
String	"Hello"	.length, .toUpperCase(), .includes()
Number	10, 3.14	.toFixed(), parseInt()
Boolean	true, false	Logical operators
Undefined	let x;	—
Null	let x = null;	—
BigInt	123n	—
Symbol	Symbol()	—

Example:

```
let name = "Prince";  
console.log(name.toUpperCase()); // PRINCE
```

✓ 2. Non-Primitive (Reference Type)

- Object
- Array
- Function

```
let arr = [1,2,3];  
arr.push(4); // method
```

```
let obj = {name: "Prince"};
```

◆ 2. DOM Manipulation

DOM = Document Object Model

It allows JavaScript to manipulate HTML.

◆ Selecting Elements (All Methods)

By ID

```
document.getElementById("box");
```

By Class

```
document.getElementsByClassName("item");
```

By Tag

```
document.getElementsByTagName("div");
```

Query Selector (First Match)

```
document.querySelector(".item");
```

Query Selector All (All Matches)

```
document.querySelectorAll(".item");
```

💡 Interview Tip:

- `getElementById()` → returns single element
 - `getElementsByClassName()` → HTMLCollection
 - `querySelectorAll()` → NodeList
-

◆ 3. Add & Remove Class

Add Class

```
element.classList.add("active");
```

Remove Class

```
element.classList.remove("active");
```

Toggle Class

```
element.classList.toggle("active");
```

◆ 4. Add CSS Using DOM

Method 1: Inline Style

```
element.style.color = "red";  
element.style.backgroundColor = "yellow";
```

Method 2 (Best Practice): Add Class

```
element.classList.add("highlight");
```

◆ 5. Events

Events are actions performed by user.

Event	Example
click	Button click
mouseover	Hover
keydown	Keyboard
submit	Form

Example:

```
btn.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

◆ 6. CSS Selectors (Important for Interview)

◆ Basic Selectors

- `#id`
 - `.class`
 - `div`
 - `*` (Universal)
-

◆ Child Selector

Selects **direct child only**

```
div > p {  
  color: red;  
}
```

Example:

```
<div>  
  <p>Selected</p>  
  <span>  
    <p>Not Selected</p>  
  </span>  
</div>
```

Only first `<p>` is selected.

◆ Sibling Selectors

1 Adjacent Sibling (+)

Selects immediate next sibling

```
h1 + p {  
  color: blue;  
}
```

Only first **p** after **h1**.

2 General Sibling (~)

Selects all siblings after

```
h1 ~ p {  
  color: green;  
}
```

All **p** after **h1**.

◆ 7. Pseudo Selectors

◆ Pseudo Classes

Used for state.

```
a:hover { color: red; }  
input:focus { border: 2px solid blue; }  
li:first-child { color: green; }
```

◆ Pseudo Elements

Used for styling part of element.

```
p::first-letter {  
  font-size: 30px;  
}
```

Other examples:

- `::before`
 - `::after`
 - `::first-line`
-

◆ 8. Custom Font Include

Method 1: Google Fonts (Easy Way)

```
<link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap" rel="stylesheet">
```

Method 2: @font-face

```
@font-face {  
  font-family: "MyFont";  
  src: url("myfont.woff2") format("woff2"),  
       url("myfont.woff") format("woff");  
}
```

```
body {  
  font-family: "MyFont";  
}
```

◆ 9. Font Types

Type	Meaning
WOFF	Web Open Font Format
WOFF2	Compressed version (Best for web)
TTF	TrueType Font

💡 Interview Answer:

- WOFF2 → Best for modern browsers
 - TTF → Bigger file size
 - WOFF → Standard web font
-

Interview Summary Answer (Short Version)

If interviewer asks:

👉 “Difference between child and sibling selector?”

You answer:

- `>` selects direct child only.
 - `+` selects immediate next sibling.
 - `~` selects all siblings after.
-

If you want, I can now give:

- 🔥 Scenario-Based Frontend Questions
- 🔥 Real Project-Based Interview Questions
- 🔥 Advanced DOM Trick Questions

Tell me which level you want 😎

j query

jQuery – Interview Guide (Simple + Professional Explanation)

Since you're strong in JavaScript, think of **jQuery** as:

A fast, small JavaScript library that simplifies DOM manipulation, events, animations, and AJAX.

Official Website: [jQuery](#)

◆ 1. Why jQuery Was Created?

Before modern ES6+ JavaScript:

- Different browsers behaved differently
- DOM manipulation was complex
- AJAX was difficult

jQuery solved:

- Cross-browser compatibility
 - Short syntax
 - Easy animations
 - Easy AJAX
-

◆ 2. How to Include jQuery

CDN Method

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

◆ 3. Basic Syntax

```
$(selector).action();
```

Example:

```
$("#box").hide();
```

Here:

- `$` → jQuery
 - `#box` → selector
 - `.hide()` → action
-

◆ 4. Document Ready

Ensures DOM loads first.

```
$(document).ready(function(){  
    console.log("DOM Loaded");  
});
```

Shortcut:

```
$(function(){  
    console.log("DOM Loaded");  
});
```

◆ 5. Selectors in jQuery

Same as CSS selectors:

Selector	Example
ID	<code>\$("#id")</code>
Class	<code>\$(".class")</code>

Tag	<code>\$("div")</code>
Attribute	<code>\$("[type='text']")</code>

◆ 6. DOM Manipulation

Change Text

```
$("#demo").text("Hello");
```

Change HTML

```
$("#demo").html("<b>Hello</b>");
```

Change CSS

```
$("#demo").css("color", "red");
```

Add Class

```
$("#demo").addClass("active");
```

Remove Class

```
$("#demo").removeClass("active");
```

Toggle Class

```
$("#demo").toggleClass("active");
```

◆ 7. Events in jQuery

Click Event

```
$("#btn").click(function(){  
    alert("Clicked!");  
});
```

Recommended Way (Better for Interview)

```
$("#btn").on("click", function(){
```

```
    alert("Clicked!");  
});
```

Why `.on()`?

- Supports event delegation
 - Better for dynamic elements
-

◆ 8. Effects & Animations

```
$("#box").hide();  
$("#box").show();  
$("#box").toggle();
```

Fade Effects

```
$("#box").fadeIn();  
$("#box").fadeOut();
```

Slide Effects

```
$("#box").slideUp();  
$("#box").slideDown();
```

◆ 9. AJAX in jQuery

```
$.ajax({  
  url: "data.json",  
  method: "GET",  
  success: function(response){  
    console.log(response);  
  }  
});
```

Shortcut:

```
$.get("data.json", function(data){  
  console.log(data);  
});
```

◆ 10. jQuery vs JavaScript (Interview Answer)

Feature	jQuery	Modern JS
Syntax	Short	Slightly longer
Browser Support	Excellent	Now also good
Performance	Slightly slower	Faster
Today Usage	Legacy projects	Modern projects

Relative → Move from where it already is, but space stays there.

Absolute → Remove it from layout and place it exactly where you want.

Relative keeps its space.

Absolute does not keep its space.